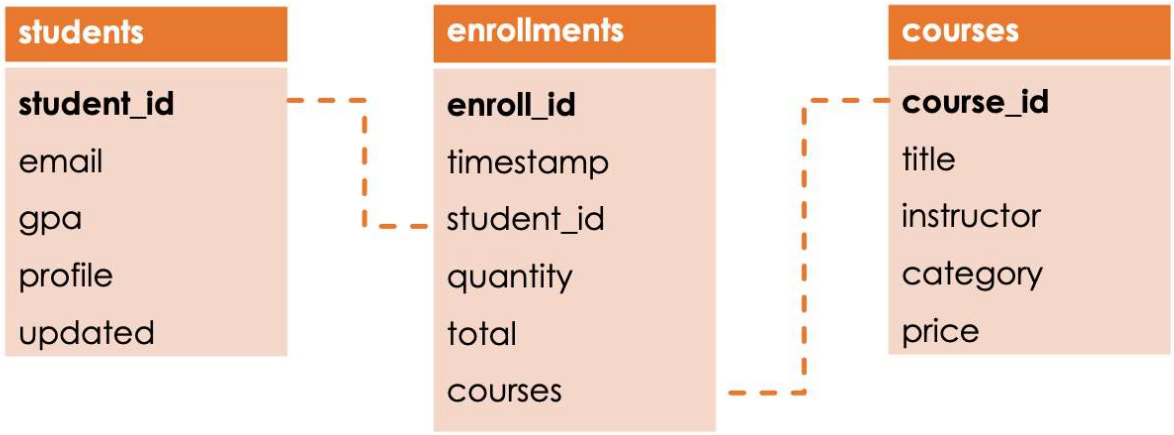
(https://databricks.com)

Lab Solution: Multi-Hop Architecture



Run the following cell to setup the lab environment

```
%run ../Includes/Setup-Lab
```

Q1- Declaring Bronze Table

Use Auto Loader to incrementally load enrollments json files from the directory **{dataset_school}/enrollments-json-raw** to a Delta table called `bronze`

```
dataset_source = f"{dataset_school}/enrollments-json-raw"
bronze_checkpoint_path = "dbfs:/mnt/DE-Associate/checkpoints/school/bronze"
schema_location = bronze_checkpoint_path

# ANSWER
(spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", schema_location)
        .load(dataset_source)
      .writeStream
        .option("checkpointLocation", bronze_checkpoint_path)
        .outputMode("append")
        .table("bronze")
)
```

Let's create a streaming temporary view from the bronze table in order to perform transformations using SQL.

```
9

(spark
  .readStream
  .table("bronze")
  .createOrReplaceTempView("bronze_tmp"))
```

Q2 - Data Cleansing & Enrichment

Using CTAS syntax, define a new streaming view `bronze_cleaned_tmp` against `bronze_tmp` that does the following:

- Remove records with **quantity** of 0 item
- Add a column called `processing_time` containing the current timestamp using the **current_timestamp()** function

```
11

%sql
-- ANSWER
CREATE OR REPLACE TEMPORARY VIEW bronze_cleaned_tmp AS
SELECT
  *, current_timestamp() processing_time
  FROM bronze_tmp
  WHERE quantity > 0
```

Q3 - Declaring Silver Table

Stream the data from `bronze_cleaned_tmp` to a table called `silver`.

```
13

silver_checkpoint_path = "dbfs:/mnt/DE-Associate/checkpoints/school/silver"

# ANSWER
(spark.table("bronze_cleaned_tmp")
              .writeStream
              .option("checkpointLocation", silver_checkpoint_path)
              .outputMode("append")
              .table("silver")
)
```

Let's create a streaming temporary view from the silver table in order to perform business-level aggregation using SQL

```
15
(spark
  .readStream
  .table("silver")
  .createOrReplaceTempView("silver_tmp"))
```

Q4- Declaring Gold Table

Using CTAS syntax, define a new streaming view `enrollments_per_student_tmp_vw` against `silver_tmp` to count the number of enrollments per `student`. Name the aggregated field: `enrollments_count`

```
17
%sql
-- ANSWER
CREATE OR REPLACE TEMPORARY VIEW enrollments_per_student_tmp_vw AS
SELECT
  student_id, count(enroll_id) AS enrollments_count
  FROM silver_tmp
  GROUP BY student_id
```

Stream the aggregated data from the `enrollments_per_student_tmp_vw` view to a Delta table called `gold_enrollments_stats`.

```
19
gold_checkpoint_path = "dbfs:/mnt/DE-Associate/checkpoints/school/gold_enrollments_stats"

# ANSWER
query = (spark.table("enrollments_per_student_tmp_vw")
              .writeStream
              .option("checkpointLocation", gold_checkpoint_path)
              .outputMode("complete")
              .table("gold_enrollments_stats"))
```

Query the data in the `gold_enrollments_stats` table to ensure data was written as expected.

```
21
%sql
SELECT * FROM gold_enrollments_stats
```

Run the below cell to land new a json file of enrollments data

```
23
load_new_json_data()
```

Wait for the new data to be propagated, and then run the below query to verify that the statistics have been updated in the table **gold_enrollments_stats**

```
25
%sql
SELECT * FROM gold_enrollments_stats
```

Finally, run the below cell for canceling the above streaming queries

27

```
for s in spark.streams.active:
    print("Stopping stream: " + s.id)
    s.stop()
    s.awaitTermination()
```

28